

ЛР5 (3 пары / 6 часов)

«Редактирование задачи: изменить название и описание. Валидация. Обновление JSON»

Что уже умеет проект

- ✓ Добавлять задачи
- ✓ Показывать список
- ✓ Менять статус
- ✓ Удалять
- ✓ Сохранять и загружать из `data/tasks.json`

Что добавляем в ЛР5

1. Добавляем поле **Description** (описание) в задачу
2. Добавляем пункт меню: **Редактировать задачу**
3. Реализуем:
 - изменить **Title** (обязательное, не пустое)
 - изменить **Description** (необязательное)

Почему так: это настоящий "Update" из CRUD, и детям очень понятно, зачем это.

0) Итог ЛР5 (что должно работать)

- ✓ В меню есть пункт "Редактировать задачу"
- ✓ Можно выбрать задачу по Id и:
 - поменять название
 - поменять описание

- ✓ После редактирования изменения **сохраняются в JSON**
 - ✓ После перезапуска изменения **не пропадают**
 - ✓ Программа не падает на ошибках ввода
-

1) Теория (важная и простая)

1.1. Что такое “редактирование” (Update)

Это операция, когда объект уже существует, и мы меняем его поля.

Пример:

- было: “Купить тетрадь”
- стало: “Купить тетрадь и ручку”

Важно:

- редактируем **по Id**, потому что название может повторяться.
-

1.2. Почему Description можно пустым

Описание — это доп. информация. Оно не обязательно.

Правило:

- **Title** — обязательно
 - **Description** — можно оставить пустым
-

1.3. Почему JSON “сам обновится”

Когда мы добавим новое поле **Description** в модель, сериализация в JSON просто начнёт записывать его.

А если в старом JSON нет **Description** — при загрузке оно будет просто пустым (""). Это нормально.

2) Практика. Пара 1: добавляем Description в модель и показываем в списке

Шаг 1. Открыть решение

1. Visual Studio → открыть `TaskTracker.sln`

Шаг 2. Добавить поле Description в TaskItem

1. В Solution Explorer откройте:

`TaskTracker.Core → Models → TaskItem.cs`

2. Найдите класс и добавьте поле `Description`

Должно получиться так:

```
namespace TaskTracker.Core.Models;

public class TaskItem
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public string Description { get; set; } = "";    // ← добавили
    public TaskStatus Status { get; set; } = TaskStatus.New;
}
```

✓ Сохранить файл (Ctrl+S)

Шаг 3. Обновить вывод списка (чтобы было видно описание)

Откройте `TaskTracker.App → Program.cs`.

Найдите место, где выводится список (пункт "2").

Замените строку вывода на такую:

```
Console.WriteLine($"{t.Id}. {t.Title} [{t.Status}]");
if (!string.IsNullOrEmpty(t.Description))
    Console.WriteLine($"    Описание: {t.Description}");
```

Это “две строки”, зато детям очень понятно.

Шаг 4. Проверка сборки

1. **Build** → **Build Solution**
2. Должно быть **Build succeeded**

Шаг 5. Коммит №1 (Description добавлен)

Git Changes → сообщение:

- `feat: add Description field to TaskItem and show it in list`

Commit All → Push.

3) Практика. Пара 2: добавляем редактирование в Core (TaskService)

Шаг 6. Открыть TaskService

`TaskTracker.Core → Services → TaskService.cs`

Шаг 7. Добавить метод редактирования

Внутри класса `TaskService` добавьте метод:

```
public TaskItem Update(int id, string newTitle, string newDescription)
{
```

```
        if (string.IsNullOrEmpty(newTitle))
            throw new ArgumentException("Название не может быть пустым.");

        var task = GetExisting(id);

        task.Title = newTitle.Trim();
        task.Description = (newDescription ?? "").Trim();

        return task;
    }
```

✓ Сохранить файл (Ctrl+S)

Шаг 8. (ВАЖНО) Обновить Add, чтобы Description не терялся

Сейчас в Add вы создаёте задачу без Description. Это нормально, но лучше добавить ввод позже (в App).

В Core Add можно оставить как есть — `Description` будет пустой.

Ничего не меняем здесь (для простоты).

Шаг 9. Сборка

Build → Build Solution → должно быть Build succeeded.

Шаг 10. Коммит №2 (Core: Update)

Git Changes → сообщение:

- `feat: add Update method to TaskService`

Commit All → Push.

4) Практика. Пара 3: добавляем пункт меню "Редактировать задачу" + сохранение в JSON + отчёт

Шаг 11. Добавить новый пункт меню

Откройте `Program.cs` и в меню добавьте строку:

- `5) Редактировать задачу`

Например так:

```
Console.WriteLine("5) Редактировать задачу");
```

Шаг 12. Добавить обработку пункта 5

Найдите в `while(true)` места, где у вас `if (input == "1")`, `if (input == "2")` и т.д.

Добавьте новый блок:

```
if (input == "5")
{
    var tasks = service.GetAll();
    if (tasks.Count == 0)
    {
        Console.WriteLine("Список задач пуст. Нечего редактировать.");
        continue;
    }

    Console.WriteLine("Список задач:");
    foreach (var t in tasks)
    {
        Console.WriteLine($"{t.Id}. {t.Title} [{t.Status}]");
        if (!string.IsNullOrEmpty(t.Description))
            Console.WriteLine($"    Описание: {t.Description}");
    }
}
```

```

n}");
    }

    if (!TryReadInt("Введите Id задачи для редактирования: ",
out var id))
    {
        Console.WriteLine("Ошибка: Id должно быть числом.");
        continue;
    }

    Console.Write("Введите новое название (Title): ");
    var newTitle = Console.ReadLine() ?? "";

    Console.Write("Введите новое описание (можно пусто): ");
    var newDescription = Console.ReadLine() ?? "";

    try
    {
        var updated = service.Update(id, newTitle, newDescription);

        // Сохраняем в JSON после изменения
        storage.Save(service.GetAll());

        Console.WriteLine("Задача обновлена:");
        Console.WriteLine($"{updated.Id}. {updated.Title} [{updated.Status}]");
        if (!string.IsNullOrEmpty(updated.Description))
            Console.WriteLine($"    Описание: {updated.Description}");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine("Ошибка: " + ex.Message);
    }
}

```

```
        continue;  
    }
```

Важно: здесь мы используем `storage.Save(...)`. Убедитесь, что в `Program.cs` у вас есть `storage` как в ЛР4.

Шаг 13. Обновить добавление задачи (необязательно, но полезно)

Чтобы было интереснее, можно попросить вводить описание уже при добавлении (не обязательно для зачёта ЛР5).

Если хотите — замените “Добавить задачу” на:

- ввод Title
- ввод Description

И сделайте `service.Add` прежним, а Description задавайте позже через `Update` — но это лишнее.

Для слабой группы лучше оставить “описание задаём через редактирование”.

Шаг 14. Проверка (F5) — обязательный сценарий

Запустите (F5) и проверьте:

Сценарий 1 (норма)

1. Добавить задачу “Купить тетрадь”
2. Редактировать (5) → выбрать `Id=1`
 - Title: “Купить тетрадь и ручку”
 - Description: “В магазине у дома”
3. Показать список (2) → должно отображаться описание
4. Выйти (0)
5. Запустить снова → список должен сохраниться (и описание тоже)

Сценарий 2 (ошибки)

1. Редактировать (5) → Id = `abc` → "Id должно быть числом"
2. Редактировать (5) → Id = `999` → "не найдена"
3. Редактировать (5) → Title пустой → "Название не может быть пустым"

Шаг 15. Отчёт `docs/LR5_Report.md`

Создайте файл `docs/LR5_Report.md` и вставьте:

```
# Отчёт ЛР5 – TaskTracker

## Что сделано
- В модель TaskItem добавлено поле Description
- В TaskService добавлен метод Update(id, newTitle, newDescription)
- В консольное меню добавлен пункт 5) Редактировать задачу
- После редактирования данные сохраняются в data/tasks.json

## Как проверить
1) Запустить (F5)
2) Добавить задачу
3) Редактировать задачу: поменять Title и Description
4) Показать список – видно описание
5) Перезапустить программу – изменения сохранились
```

Шаг 16. Коммит №3 (App: редактирование + отчёт)

Git Changes → сообщение:

- `feat: add edit task command and LR5 report`

Commit All → Push.

5) Что сдавать (чек-лист)

Студент сдаёт:

✓ ссылку на GitHub

✓ `docs/LR5_Report.md`

✓ видео/скрин 30–60 сек:

- добавил задачу
- отредактировал (Title+Description)
- перезапуск → изменения остались

✓ минимум 3 коммита за ЛР5

6) Частые проблемы и быстрые решения

Проблема: “storage не виден в Program.cs”

Проверьте, что в Program.cs есть строки из ЛР4:

```
var storage = new JsonTaskStorage(dataFilePath);
```

и добавлен using:

```
using TaskTracker.Storage.Services;
```

Проблема: “После редактирования не сохраняется”

Проверьте, что после `service.Update(...)` есть:

```
storage.Save(service.GetAll());
```

Проблема: “Описание не видно”

Проверьте, что в выводе списка есть блок:

```
if (!string.IsNullOrEmpty(t.Description))
```

```
Console.WriteLine($"    Описание: {t.Description}");
```
